

CSC 452/552 Operations Systems

Project 3 Threads

Name: Sean Varie

Bronco ID: 114159195

Date: November 12, 2024

1. Project Overview

Project 3 was to implement a multithreaded mergesort algorithm. In addition to the multithreaded implementation of mergesort, a script was written to run a driver class which sorts an array with a given number of threads and reports the time taken to sort it. The script runs this driver with a given size of array and a varying number of threads from 1 to 32. The outputted times are then plotted on a graph.

2. Project Management Plan

a) Task 3: Make Thread safe

Task 3 was the implementation of multithreaded mergesort. I went through a few iterations of this implementation before coming to my final result. This implementation involved splitting the array into (mostly) equal parts, with the first section getting any extra, and then handing each section off to a thread to sort, before merging all the sections together. The instructions did specify the exact algorithm to use in order to merge the sections of the array which have been sorted by the other threads. To accomplish this, I wrote an algorithm that repeatedly goes through the first element in every section and finds the smallest one, until there are no elements left. This algorithm, on its own, is faster than any algorithm which could be implemented to accomplish the same goal using the provided `merge_s` function*. The worst case runtime of this algorithm (just the merging after all child threads have completed, not the whole sorting algorithm) is $O(n*t)$ where n is the size of the array and t is the number of sorted sections, or the number of threads. One of the other implementations I had was much more similar to the actual mergesort algorithm, except that it sometimes creates new threads to handle the recursive calls. This does not meet the requirements of the project, as it does not split the list among the threads evenly (unless the number of threads is a power of 2). Despite this, I have included this implementation in my `lab.c` file under the name `"true_mergesort_mt"`, as the results of this implementation provide some interesting insights into the effect of threading on the performance of this algorithm.

*The algorithm used for merging the sorted arrays after the threads have finished

b) Task 4: Driver app

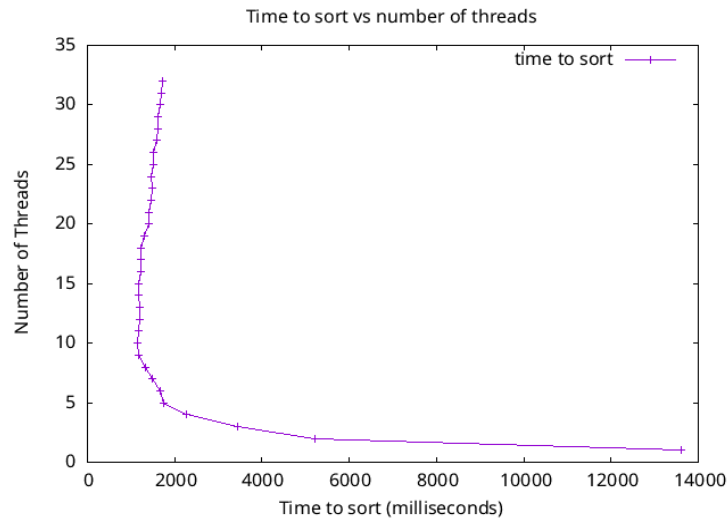
For task 4, I used the provided driver to experiment with sorting different sizes of arrays with different numbers of threads.

c) Task 5: Add bash files

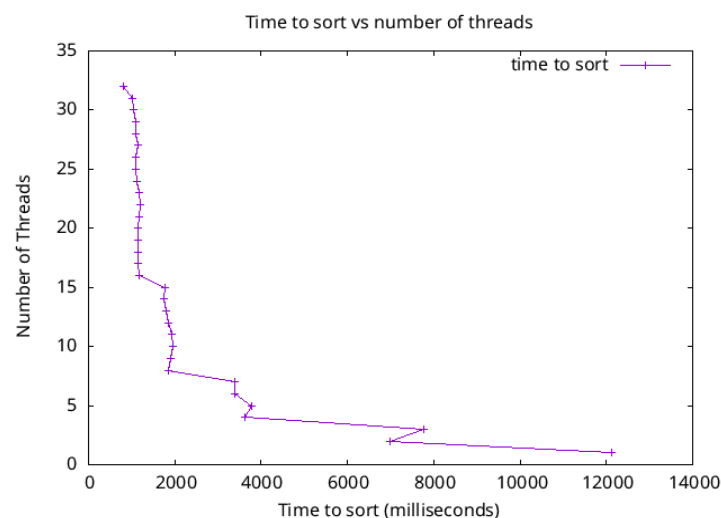
For task 5, I stopped using github codespaces and moved to my local machine in order to avoid difficulties installing packages to use in my scripts. I installed the packages specified in the instructions and used the provided script as a starting

point, modifying it to run my program. I ran the script to generate a plot of the time taken to sort a 10000000 integer array with a different number of threads. I have included this plot, as well as the raw data in my submission. I have also included a plot that was generated the same way except using the `true_mergesort_mt` function. The comparisons of these plots provide some valuable insights into what is the thread's effect on efficiency.

d) Task 6: Complete the Analysis



sort time plot for mergesort_mt



sort time plot for true_mergesort_mt

As can be seen above, I was able to generate a plot very similar to the example shown in the project instructions with my final implementation of the threaded mergesort (`mergesort_mt`). In the graph for `mergesort_mt` you can see a slow down similar to the provided example. The optimal number of threads for the particular run shown was 10, though the times for all the runs from 9 threads to 19 were quite close. After 19 threads, the times for `mergesort_mt` began slowing down. The slowest sorting time occurred with 1 thread.

Overall, for `mergesort_mt`, adding more threads did seem to increase the speed up until that 9 to 19 threads range, which had about the same times, and after which the algorithm began to slow. I do not believe that this is due to the overhead of

creating and context switching between threads however. As mentioned in task 3, the computation cost of merging the sorted sections after the child threads complete is $O(n*t)$ where n is the number of integers in the entire array and t is the number of threads. I believe that the slowdown above 19 threads is due to the added cost of t in the runtime equation becoming larger and, around 19 threads, more significant than the added value of an increased number of threads. This is supported by the times shown in the second plot above for `true_mergesort_mt`. This algorithm does not have a single thread handling a $O(n*t)$ cost merging of multiple sections and as far as the data in the graph goes (and as far as I experimented before I was worried about damaging my computer due to extended thermal throttle), there is no point at which more threads generally means worse performance. There are local instances of an increase of a few threads leading to worse performance, most notably at the powers of 2 (2, 4, 8, 16, etc.), i.e. 3 threads is worse than 2, which makes sense, as when the number of threads is not a power of 2, the array doesn't get split evenly. There also seems to be diminishing returns as more threads get added. Despite these factors, in general, for `true_mergesort_mt`, more threads does appear to provide faster sort times, even if only by a small amount. This testing is not extensive enough to fully accept that conclusion (which is certainly not true with an infinitely increasing number of threads). As mentioned above, I did not want to push my computers too far past its limits, as would have been required for extensive enough testing to draw a more meaningful conclusion. Despite this, it is valuable to know that, at least of my machine, a modern, but not extremely powerful, laptop, the overhead of this relatively low number of threads is not a major concern for performance, although it is a major concern for fan noise and likely power consumption.

3. Project Deliveries

a) How to compile and use my code?

To compile, run the following command from the project directory:

```
$ make
```

After compiling, run the program with the following command:

```
$ ./myprogram <array_size> <num_threads>
```

To create a plot, ensure gnuplot is installed, then run the following command:

```
$ ./createplot.sh -s <array_size> -f <output_filename>
```

To run the test suite, run the following command:

```
$ make check
```

b) Any self-modification?

I modified the provided `lab.c` file as described above to implement the multithreaded

mergesort algorithms. I also added a call to free to the driver file in order to stop a memory leak.

c) Summary of Results.

see section 2 subsection d, "Task 6: Complete the Analysis", above,

4. Self-Reflection of Project

This was an interesting project, the actual coding implementation was not very difficult, although there were a few different approaches that I went through. The script was mostly provided to us. Running the scripts and viewing and analyzing the results was very enlightening. Seeing the efficiency gains of multithreading visualized so well in the graphs above is extremely helpful. The comparison of the two algorithms I made in the analysis was also very interesting.